

# TASK-1

```
keerthi_priya@LAPTOP-OSGV x + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ cd week3
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ pwd
/home/keerthi_priya/week3
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc task1.c -o task1
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ./task1
myshell> hello
You entered: hello
myshell> linux
You entered: linux
myshell> exit
Exiting shell...
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano File1.txt
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ cat File1.txt
Hello, this is File 1.
This content will be copied to File 2.
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];

    while (1)
    {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }
        printf("You entered: %s\n", input);
    }

    return 0;
}
```

## TASK-2

```
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc task2.c -o task2
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ./task2
File copied successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ cat File2.txt
Hello, this is File 1.
This content will be copied to File 2.
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("File1.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening File1.txt");
        return 1;
    }

    destination = open("File2.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination < 0)
    {
        perror("Error opening File2.txt");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```

# TASK-3

```
keerthi_priya@LAPTOP-OSGV x + v
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc task2.c -o task2
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ./task2
File copied successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ cat File2.txt
Hello, this is File 1.
This content will be copied to File 2.
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano task3.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc task3.c -o task3
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ./task3

Enter command:
You entered:
hello
Enter command: hello
You entered: hello
hellp
Enter command: hellp
You entered: hellp
exit
Enter command: exit
You entered: exit
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <termios.h>

int main()
{
    char buffer[100];
    int position = 0;
    char ch;

    printf("Enter command: ");

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n')
        {
            buffer[position] = '\0';

            printf("\nYou entered: %s\n", buffer);

            if (position == 4 &&
                buffer[0] == 'e' &&
                buffer[1] == 'x' &&
                buffer[2] == 'i' &&
                buffer[3] == 't')
            {
                break;
            }

            position = 0;
            printf("Enter command: ");
        }
        else if (ch == 127)
        {
            if (position > 0)
```

```
keerthi_priya@LAPTOP-OSGV  ×  +  ∨
GNU nano 8.7.1 task3.c *
printf("\nYou entered: %s\n", buffer);

if (position == 4 &&
    buffer[0] == 'e' &&
    buffer[1] == 'x' &&
    buffer[2] == 'i' &&
    buffer[3] == 't')
{
    break;
}

position = 0;
printf("Enter command: ");
}
else if (ch == 127)
{
    if (position > 0)
    {
        position--;
        printf("\b\b");
    }
}
else
{
    if (position < 99)
    {
        buffer[position] = ch;
        position++;

        printf("%c", ch);
    }
}

return 0;
}
```

## TASK-4

```
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ pwd
/home/keerthi_priya/week3
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c task3 task3.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano task4.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc task4.c -o task4
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ./task4
myshell> hello
You entered: hello
myshell> keerthi
You entered: keerthi
myshell> how are you jhansi
You entered: how are you jhansi
myshell> I am fine
You entered: I am fine
myshell> exit
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define MAX_INPUT 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_INPUT];

int history_count = 0;
int history_index = 0;

void clear_line()
{
    printf("\033[2K\r");
    printf("mysHELL> ");
}

int main()
{
    char buffer[MAX_INPUT];
    int index;
    char ch;

    struct termios oldt, newt;

    /* Save original terminal settings */
    tcgetattr(STDIN_FILENO, &oldt);

    /* Create new terminal settings */
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);

    /* Apply new settings */
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        printf("mysHELL> ");
        fflush(stdout);

        index = 0;
        buffer[0] = '\0';
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

        while (1)
        {
            read(STDIN_FILENO, &ch, 1);

            /* ENTER */
            if (ch == '\n' || ch == '\r')
            {
                buffer[index] = '\0';
                printf("\n");

                break;
            }

            /* BACKSPACE */
            else if (ch == 127 || ch == 8)
            {
                if (index > 0)
                {
                    index--;
                    printf("\b \b");
                    fflush(stdout);
                }
            }

            /* ESCAPE SEQUENCE */
            else if (ch == 27)
            {
                char seq1;
                char seq2;

                read(STDIN_FILENO, &seq1, 1);
                read(STDIN_FILENO, &seq2, 1);

                /* UP ARROW */
                if (seq1 == '[' && seq2 == 'A')
                {
                    if (history_count > 0 && history_index > 0)
                    {
                        history_index--;
                        strcpy(buffer, history[history_index]);
                    }
                }
            }
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

        index = strlen(buffer);
        clear_line();
        printf("%s", buffer);
        fflush(stdout);
    }
}
/* DOWN ARROW */
else if (seq1 == '[' && seq2 == 'B')
{
    if (history_index < history_count - 1)
    {
        history_index++;
        strcpy(buffer, history[history_index]);
        index = strlen(buffer);
        clear_line();
        printf("%s", buffer);
        fflush(stdout);
    }
    else
    {
        history_index = history_count;
        buffer[0] = '\0';
        index = 0;
        clear_line();
        fflush(stdout);
    }
}
/* NORMAL CHARACTER */
else
{
    |
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

    }
}
/* NORMAL CHARACTER */
else
{
    if (index < MAX_INPUT - 1)
    {
        buffer[index] = ch;
        index++;
        write(STDOUT_FILENO, &ch, 1);
    }
}
/* EXIT */
if (strcmp(buffer, "exit") == 0)
{
    break;
}
/* STORE COMMAND IN HISTORY */
if (strlen(buffer) > 0)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], buffer);
        history_count++;
    }
    history_index = history_count;
}
printf("You entered: %s\n", buffer);
}
/* Restore original terminal settings */
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
return 0;
}
|
```

## TASK-5

```
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ nano task5.c
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ gcc task5.c -o task5
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$ ./task5
myshell> hello
You entered: hello
myshell> keerthi
You entered: keerthi
myshell> exit
keerthi_priya@LAPTOP-OSGVCE7A:~/week3$
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node
{
    char *command;
    struct Node *next;
} Node;

Node *head = NULL;

/* Add a command to the linked list */
void add_history(const char *command)
{
    Node *new_node;

    new_node = malloc(sizeof(Node));

    if (new_node == NULL)
    {
        perror("malloc failed");
        exit(1);
    }

    new_node->command = malloc(strlen(command) + 1);

    if (new_node->command == NULL)
    {
        perror("malloc failed");
        free(new_node);
        exit(1);
    }

    strcpy(new_node->command, command);

    new_node->next = NULL;

    if (head == NULL)
    {
        head = new_node;
    }
}
```



keerthi\_priya@LAPTOP-OSGV



GNU nano 8.7.1

```
        head = new_node;
    }
    else
    {
        Node *current = head;

        while (current->next != NULL)
        {
            current = current->next;
        }

        current->next = new_node;
    }
}

/* Display all stored commands */
void display_history()
{
    Node *current = head;
    int number = 1;

    while (current != NULL)
    {
        printf("%d %s\n", number, current->command);

        current = current->next;
        number++;
    }
}

/* Free the complete linked list */
void free_history()
{
    Node *current = head;

    while (current != NULL)
    {
        Node *temp = current;

        current = current->next;
```

```
        free(temp->command);
        free(temp);
    }

    head = NULL;
}

int main()
{
    char *buffer = NULL;
    size_t buffer_size = 0;
    ssize_t length;

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        length = getline(&buffer, &buffer_size, stdin);

        if (length == -1)
        {
            break;
        }

        if (length > 0 && buffer[length - 1] == '\n')
        {
            buffer[length - 1] = '\0';
        }

        if (strcmp(buffer, "exit") == 0)
        {
            break;
        }

        if (strcmp(buffer, "history") == 0)
        {
            display_history();
            continue;
        }
    }
}
```

```
{
    printf("myshell> ");
    fflush(stdout);

    length = getline(&buffer, &buffer_size, stdin);

    if (length == -1)
    {
        break;
    }

    if (length > 0 && buffer[length - 1] == '\n')
    {
        buffer[length - 1] = '\0';
    }

    if (strcmp(buffer, "exit") == 0)
    {
        break;
    }

    if (strcmp(buffer, "history") == 0)
    {
        display_history();
        continue;
    }

    if (strlen(buffer) > 0)
    {
        add_history(buffer);
    }

    printf("You entered: %s\n", buffer);
}

free(buffer);
free_history();

return 0;
}
```

